



TOPIC 5

Service-based, Event-based Systems

Big Data Modeling

Course Review

Session 1

Service-based, Event-based Systems

Big Data Modeling

A solid orange horizontal bar spanning the width of the text area below the main title.

Session 1 Overview

After this session, students should be able to:

- Explain the idea of service-based systems and SOA.
- Distinguish service-based and event-based system design.
- Describe how Web services/APIs support system integration.
- Explain why big data requires different modeling considerations.
- Identify key modeling concerns: data sources, data flow, metadata, quality, governance.

Why Modern Systems Need New Design Approaches

Modern information systems are no longer isolated applications.

They often need to:

- Communicate with other systems.
- Exchange data with external services.
- Support real-time or near-real-time business events.
- Handle large, fast, and diverse data sources.
- Scale across cloud and distributed environments.

System design is not only about screens and databases; it is also about how components communicate and how data moves across the organization.

From Monolithic Systems to Service-based Systems

Monolithic system

- One large application contains most business functions.
- Components are tightly connected.
- Changes in one function may affect many parts of the system.
- Deployment and maintenance can become difficult as the system grows.

Service-based system

- Business functions are divided into smaller services.
- Each service focuses on a specific business capability.
- Services communicate through APIs or messages.
- Easier to reuse, replace, scale, and maintain.

What Is a Service-based System?

A service-based system is a system designed as a set of independent but connected services.

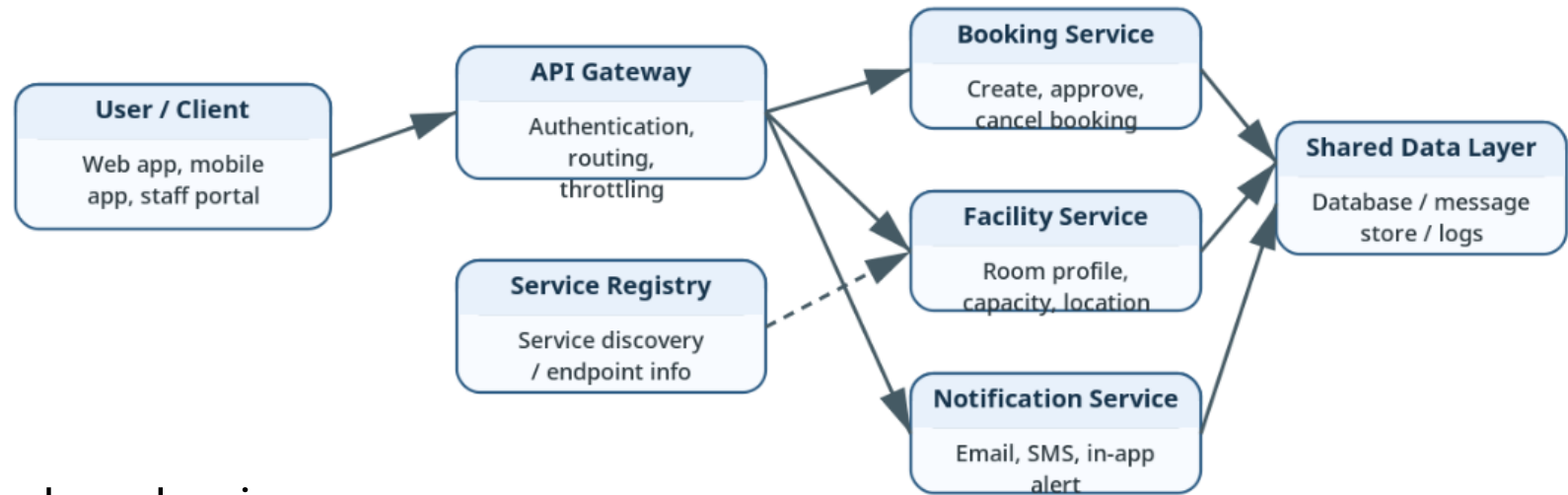
A service usually has:

- A clear responsibility.
- A defined input and output.
- A standard interface.
- Reusable business logic.
- Communication with other services through APIs or messages.

Example services in a Facility Booking System

- User Service: login, role checking.
- Facility Service: room details and availability.
- Booking Service: create and manage bookings.
- Approval Service: approve or reject requests.
- Notification Service: send email or system alerts

Service-Oriented Architecture - SOA



SOA is an architectural approach where business processes are broken down into individual services.

Main principles:

- **Reusability:** one service can be used in many applications.
- **Interoperability:** services can work with other services or systems.
- **Componentization:** each service is modular and focuses on one responsibility.

SOA helps organizations respond faster when business requirements change, but integration and orchestration can be complex.

Service-based Architecture Components

Common components:

- **Client/User Interface:** web app, mobile app, staff portal.
- **API Gateway:** receives requests, routes them to services.
- **Business Services:** booking, facility, user, approval, notification.
- **Service Registry:** stores service locations and endpoint information.
- **Database/Data Layer:** stores operational data.
- **Monitoring/Logging:** tracks service performance and errors.

The user sees one system, but internally the system may call multiple services.

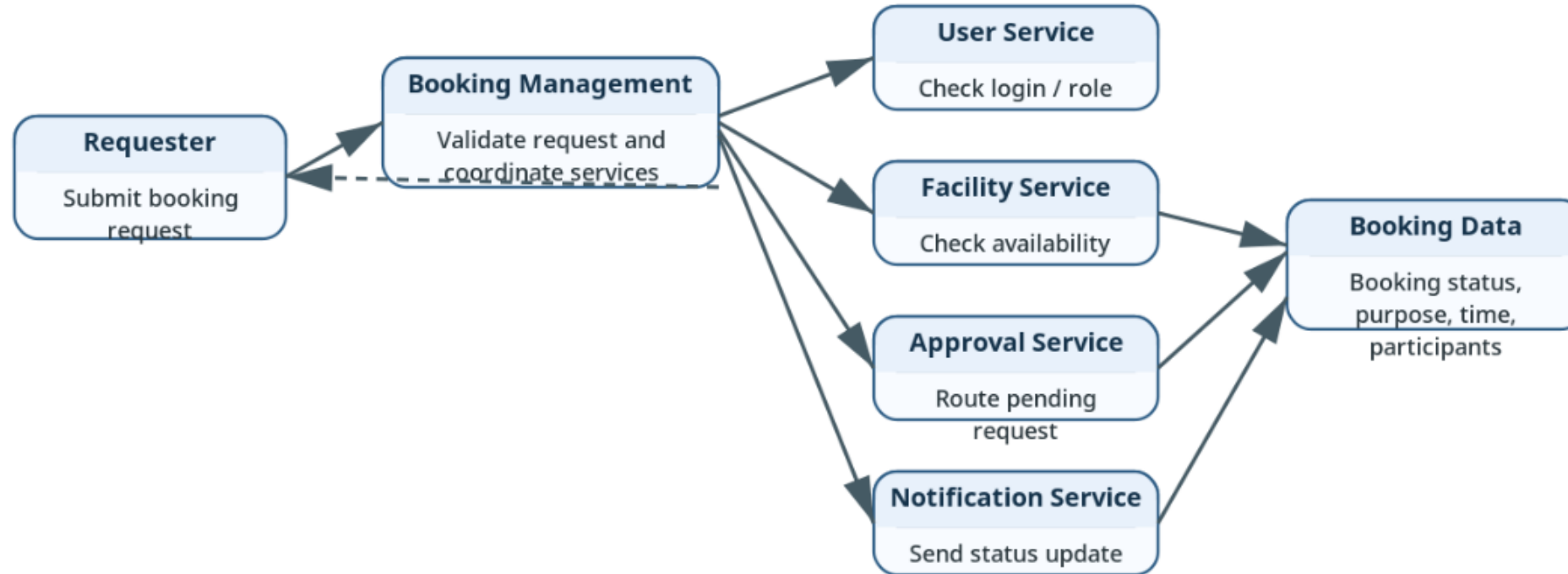
Service Orchestration

Service orchestration means one main component controls the sequence of service calls.

Example: Facility Booking Request

1. User submits booking request.
2. Booking Service validates required information.
3. User Service checks login and role.
4. Facility Service checks room availability.
5. Booking Service saves request.
6. Approval Service routes the request to staff.
7. Notification Service informs requester and staff.

Service Orchestration



Flow: Requester submit request; Booking management validates user, check facility, saves booking data, and send notification

Orchestration = one component controls the sequence of service calls

Web Services and APIs

A Web service allows two software systems to communicate over a network.

Web services help systems:

- Exchange data.
- Share business logic.
- Integrate applications built in different languages.
- Communicate across organizational boundaries.
- Use standard formats such as XML or JSON.

Example

A university booking system may call:

- Email API to send notifications.
- Calendar API to check schedules.
- Authentication API to verify users.

Web services commonly support SOA and allow systems written in different languages to exchange data over the Internet.

Benefits and Challenges of Service-based Systems

Benefits

- Easier maintenance.
- Reusable functions.
- Better scalability.
- Easier integration with external systems.
- Faster response to changing requirements.

Challenges

- More complex architecture.
- More network communication.
- Service dependency issues.
- Need for API security.
- Need for monitoring and error handling.

Service-based design improves flexibility, but students must also think about integration complexity.

What Is an Event-based System?

An event-based system reacts to events that occur in the business environment.

An event is something meaningful that happens.

Examples:

- Booking request submitted.
- Payment completed.
- Facility booking approved.
- Room booking cancelled.
- Sensor detects room occupancy.
- User fails login three times.

Event-based Architecture

Main components:

- **Event Producer:** creates and publishes an event.
- **Event Broker/Event Bus:** receives and routes events.
- **Event Consumer:** subscribes to and processes events.
- **Event Store/Log:** stores events for tracking, audit, or replay.

Example

When `BookingApproved` occurs:

- Notification Service sends email.
- Reporting Service updates usage statistics.
- Calendar Service blocks the room.
- Audit Service records the action.

Service-based vs Event-based Systems

Aspect	Service-based	Event-based
Main idea	Request and response	Publish and react
Communication	Direct service call	Indirect through events
Timing	Usually synchronous	Often asynchronous
Coupling	Service caller knows service provider	Producer does not need to know consumers
Good for	Business operations and transactions	Notifications, real-time updates, audit, analytics

These two styles can be combined in the same system.

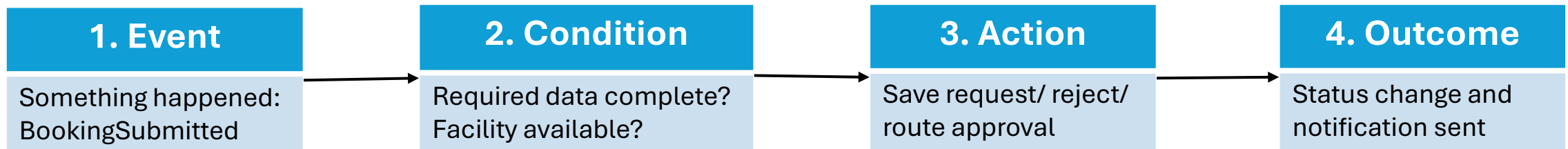
Event + Business Rule Model

An event-based rule can be documented using:

- **User rule:** the business rule to enforce.
- **Event:** what action triggers the rule.
- **Entity:** the data object affected.
- **Condition:** what must be checked.
- **Action:** what the system does when the condition is met.

Example:

- User rule: Facility cannot be double booked.
- Event: Submit booking request.
- Entity: Booking.
- Condition: requested time overlaps approved booking.
- Action: reject request and notify requester.



Event-based Scenario: Facility Booking System

Scenario flow:

1. User submits booking request.
2. System publishes `BookingSubmitted`.
3. Validation component checks required information.
4. Facility component checks availability.
5. If valid, system publishes `BookingPendingApproval`.
6. Staff approves or rejects.
7. System publishes `BookingApproved` or `BookingRejected`.
8. Notification component sends message to user.

Which parts should be service calls, and which parts should be events?

When Should We Use Event-based Design?

Use event-based design when the system needs:

- Notifications.
- Real-time updates.
- Audit trail.
- Multiple components reacting to the same business event.
- Loose coupling between components.
- Data streaming or analytics.

Examples:

- Booking status update.
- Payment confirmation.
- Stock level alert.
- Attendance check-in.
- Sensor data collection.
- Security warning.

What Is Big Data?

Big data refers to data that is difficult to manage using only traditional database approaches because of:

- **Volume:** very large amount of data.
- **Velocity:** data arrives quickly or continuously.
- **Variety:** data comes in many formats.
- **Veracity:** data quality and reliability may vary.
- **Value:** data must support useful decisions.

Big data is not only about size; it is about how data is collected, stored, processed, governed, and analyzed

Why Big Data Modeling Is Important

Traditional data modeling focuses heavily on:

- Entities.
- Attributes.
- Relationships.
- Keys.
- Constraints.

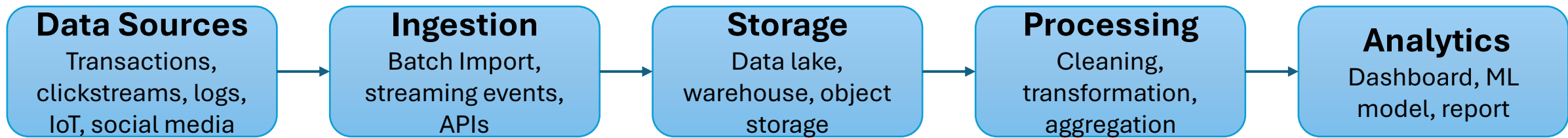
Big data modeling also needs to consider:

- Data sources.
- Data flow.
- Event data.
- Metadata.
- Data quality.
- Data retention.
- Privacy and access control.
- Analytical purpose.

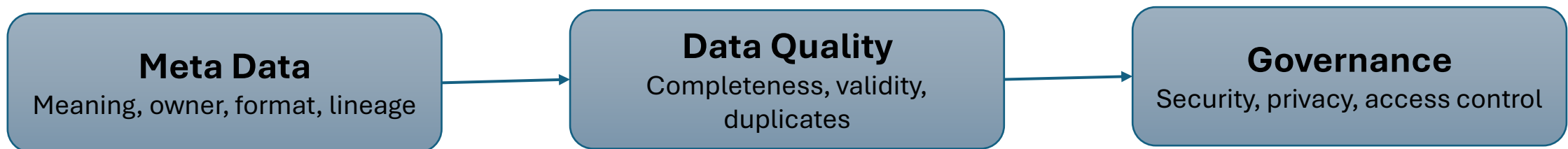
Big Data Pipeline

A big data pipeline describes how data moves from source to decision.

Typical stages:



Supporting concerns:



Modeling big data means modeling flows, meaning, quality, and governance – not only tables

Common Big Data Sources

Examples of data sources:

- Transaction data.
- System logs.
- User clickstream.
- Sensor/IoT data.
- Social media data.
- Email or text data.
- Image, video, or audio data.
- External open data.

Big data modeling starts by asking: Where does the data come from, and what decision will it support?

Data Lake, Data Warehouse, and Operational Database

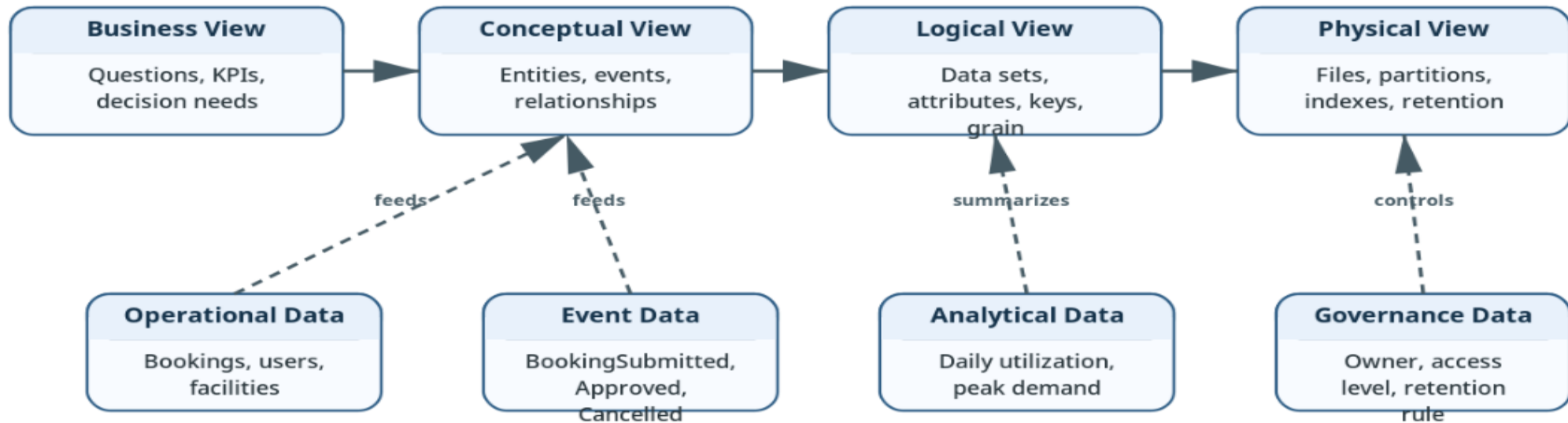
Operational database	Data warehouse	Data lake
• Supports daily transactions. • Structured data. Example: booking records, user accounts.	• Supports reporting and analysis. • Cleaned and integrated data. Example: monthly room usage report	• Stores raw or semi-structured data. • Useful for exploration and analytics. Example: logs, sensor data, clickstream

Big data modeling may combine all three.

Big Data Modeling Map

Big data modeling can be viewed from four levels:

- **Business View:** what questions and decisions are needed?
- **Conceptual View:** what entities, events, and relationships matter?
- **Logical View:** what datasets, attributes, keys, and grain are needed?
- **Physical View:** how are files, partitions, indexes, and retention designed?



Design question: What data must be retained, analyzed, protected, and traced?

Modeling Events as Data

In big data systems, events often become important analytical data.

Example event data:

- BookingSubmitted
- BookingApproved
- BookingRejected
- BookingCancelled
- FacilityCheckedIn
- RoomUsageRecorded

Important attributes:

- Event ID.
- Event type.
- Timestamp.
- Actor/User ID.
- Related entity ID.
- Status.
- Source system.

Big Data Modeling Checklist

When modeling big data, ask:

- What data sources are used?
- What business questions need answers?
- What events should be captured?
- What data should be stored raw?
- What data should be cleaned and transformed?
- What metadata is required?
- Who owns the data?
- Who can access the data?
- How long should data be retained?
- How will data quality be checked?

Session 01 Summary

Key points:

- Service-based systems divide business capabilities into reusable services.
- SOA supports reusability, interoperability, and componentization.
- Web services/APIs help systems exchange data and logic.
- Event-based systems react to meaningful business events.
- Big data modeling focuses on data sources, flows, events, metadata, quality, and governance.
- Modern system design must consider both architecture and data

Session 2

Course Review Roadmap

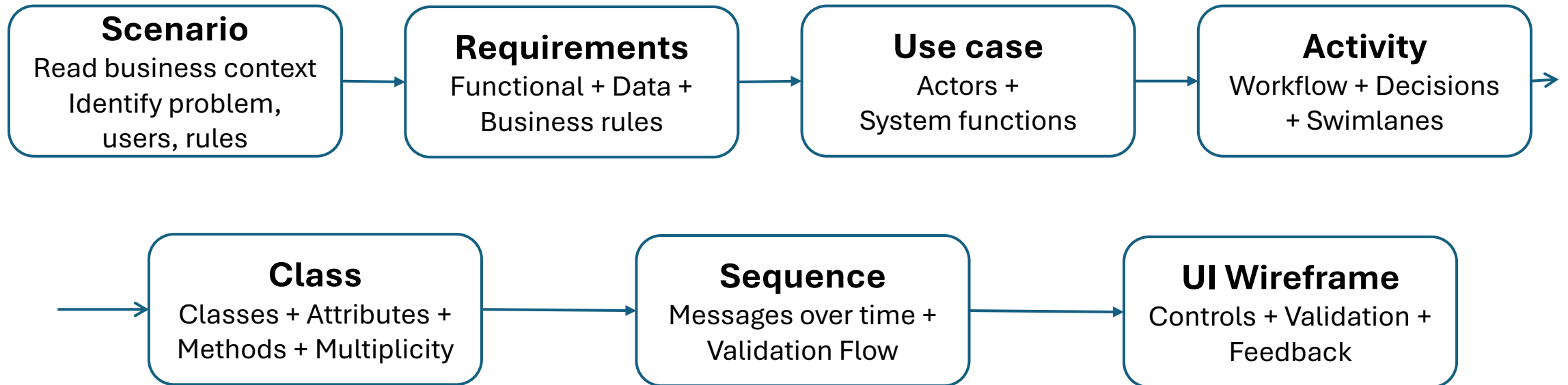


Course Review Overview

In this session, we will review:

1. Requirement Identification
2. Use Case Diagram
3. Activity Diagram
4. Class Diagram
5. Sequence Diagram
6. UI Wireframe & Controls
7. Validation and Feedback

Course Review Roadmap



All diagrams must describe the same system from different viewpoints

Final Questions Are Usually Designed

From a scenario, you should define:

- Actors/users.
- Business problems.
- Main functions.
- Data to store.
- Business rules.
- Reports/outputs.
- Exceptions.

You should underline with keyword on the scenario

Traceability Across Models

A requirement should appear consistently across multiple models.

Example:

- Requirement: User submits booking request.
- Use case: Submit Booking Request
- Activity: Enter details → Validate → Check availability → Save request
- Class: BookingRequest, User, Facility.
- Sequence: User → UI → Controller → Service → Database.
- UI: Booking Request Form.

Mini Scenario for Review

Used throughout Session 02

University Facility Booking System

Students and lecturers can log in and submit facility booking requests. The user selects facility, date, start time, end time, purpose, and expected participants. The system validates the information and checks facility availability. If valid and available, the booking is saved as pending approval. Staff review the request and approve or reject it. The system sends notification to the requester

Requirement Identification

A requirement describes what the system must do or what condition the system must satisfy.

Types:

- Functional requirement.
- Non-functional requirement.
- Data requirement.
- Business rule.
- Report/output requirement.

How to Identify Requirements from Scenario

Checklist:

- Find actors: who uses the system?
- Find verbs: what actions are performed?
- Find nouns: what data/entities are mentioned?
- Find conditions: what rules must be checked?
- Find outputs: what reports/messages are produced?
- Find exceptions: what happens if something fails?

Requirement Extraction Example from the Scenario

Scenario Clues	Functional Requirements	Data Requirements	Business Rules
• students and lecturers • log in • select facility/date/time • validate form • check availability • staff approve/reject • send notification	• Allow user login • Submit booking request • Validate required information • Check facility availability • Staff review pending requests • Approve or reject request • Send notification	• User data • Facility data • Booking request data • Approval decision • Notification record	• No double booking • Required fields must not be empty • Start time < end time • Booking date cannot be in the past • Participants must be positive

Requirement Extraction Example

From: “The user selects facility, date, start time, end time, purpose, and expected number of participants.”

To **Function requirement**:

The system shall allow users to enter booking details including facility, date, start time, end time, purpose, and expected number of participants.

Data requirement:

Booking must store facility, date, start time, end time, purpose, number of participants, status, requester, and submission date.

Requirement Writing Format

The system shall + action + object + condition.

Example:

- The system shall allow users to submit facility booking requests online.
- The system shall validate all required booking information.
- The system shall check facility availability before saving a booking request.
- The system shall allow staff to approve or reject pending booking requests.
- The system shall send notification after the booking status is updated.

Common Requirement Mistakes

- The writing is too general.
- Missing actors or users.
- Missing key data.
- Confusing requirements with technical solutions.
- Ignoring business rules.
- Failure to write verifiable requirements.

Example:

Not good: System is good; System uses database; Booking is easy.

Better : The system shall prevent double booking for the same facility at overlapping time.

Mini Exercise 1 — Identify Requirements

From the mini scenario, identify:

- 5 functional requirements.
- 3 data requirements.
- 2 business rules.

Functional requirements:

- User can log in.
- User can submit booking request.
- System validates required information.
- System checks facility availability.
- Staff can approve or reject request.
- System sends notification.

Data requirements:

- User data.
- Facility data.
- Booking request data.
- Approval data.
- Notification data.

Business rules:

- Facility cannot be double booked.
- Start time must be earlier than end time.

- Booking date cannot be in the past.

Requirement Checklist

Before moving to diagram, check:

- Did I identify all main actors?
- Did I identify all main functions?
- Did I include key data?
- Did I include business rules?
- Did I include outputs or notifications?
- Are requirements consistent with the scenario?

Use Case Diagram

A use case diagram shows system functions from the user's perspective.

It includes:

- Actor.
- Use case.
- System boundary.
- Association.
- Include.
- Extend

How to Identify Actors

Actors are external users or systems that interact with the system.

In Facility Booking System:

- Student.
- Lecturer.
- Staff.
- Manager.
- Email/Notification Service, if treated as external system

Should not draw:

- Database as actor
- Internal module as actor
- Table as actor

How to Identify Use Cases

Use cases usually come from action verbs.

Examples:

- Login.
- Submit Booking Request.
- Check Booking Status.
- Review Booking Request.
- Approve Booking Request.
- Reject Booking Request.
- Generate Usage Report

Use case name should be verb + object.

Include vs Extend

Include

Used when one use case always uses another use case.

Example:

- Submit Booking Request includes Validate Booking Information.
- Submit Booking Request includes Check Facility Availability.

Extend

Used for optional or conditional behavior.

Example:

- Send Rejection Notification extends Reject Booking Request.
- Send Approval Notification extends Approve Booking Request.

Note: Avoid overusing include/extend unless the scenario requires more detail.

Draw Use Case Diagram from the scenario

Using the mini scenario, draw a simple use case diagram with:

- At least 3 actors.
- At least 6 use cases.
- System boundary.
- Reasonable include/extend relationships.
- Time: 7 mins

Use Case Diagram Checklist

Checklist:

- Actors are outside the system boundary.
- Use cases are within the system boundary.
- Use cases are named using verbs.
- Associations are clear.
- Include/extend diagrams are not reversed.
- Do not include databases in use case diagrams.
- Do not draw too many technical details.

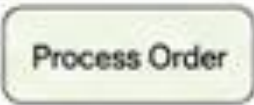
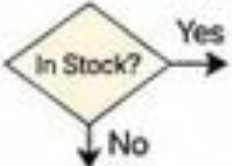
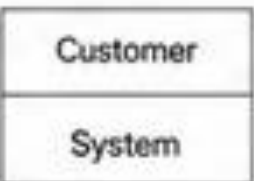
Activity Diagram

An activity diagram shows the workflow of a business process or system process.

It is useful for showing:

- Process sequence.
- Decision conditions.
- Alternative flows.
- Parallel activities.
- Responsibility by swimlane.

Activity Diagram Notation

Notation	Name	Description	Example
	Initial Node	The starting point of the activity.	
	Action / Activity	A task or processing step.	
	Control Flow	Shows the sequence or flow direction.	
	Decision Node	A point where the flow branches based on conditions.	
	Merge Node	Combines alternative flows.	
	Final Node	The ending point of the activity.	
	Swimlane	Separates activities by actor, role, or component.	

How to Build Activity Diagram from Scenario

Steps:

1. Identify start event.
2. List actions in order.
3. Identify decisions.
4. Add conditions.
5. Add alternative paths.
6. Add swimlanes if there are multiple actors.
7. End each path properly.

Mini exercise - Draw Activity Diagram

Draw an activity diagram for booking request submission and approval.

Must include:

- User swimlane.
- System swimlane.
- Staff swimlane.
- At least 2 decision nodes.
- Valid and invalid paths.
- Time: 7 mins

Activity Diagram: Common Mistakes and Checklist

Common mistakes	Checklist
No start/end statements.	Workflow in the correct order
Missing conditions on decisions	Conditional decisions clearly defined
Overlapping arrows	Complete alternative path
Activity not starting with a verb	Swimlane correctly identifying the actor/system
Incorrect responsibilities in the swimlane	Clear arrows
Drawing the database as a lane	No excessive content in a single activity
No error flow or alternative flow	Logical ending.

Class Diagram

A class diagram shows the static structure of the system.

It includes:

- Class name.
- Attributes.
- Methods.
- Relationships.
- Multiplicity.

How to Identify Classes

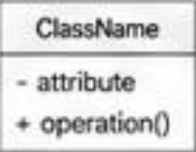
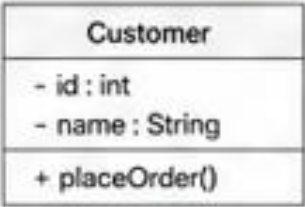
Classes often come from **important nouns** in the scenario.

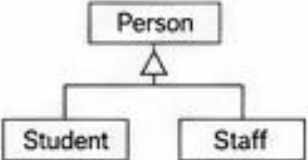
Example nouns:

- User.
- Student.
- Lecturer.
- Staff.
- Facility.
- BookingRequest.
- Approval.
- Notification.
- UsageReport.

**Not every noun becomes a class.
Only nouns with important data and roles
in the system should be selected.**

Class Structure and notation

Notation	Name	Description	Example
	Class	A blueprint for creating objects.	
	Association	A relationship between classes.	
	Aggregation	A weak whole-part relationship.	
	Composition	A strong whole-part relationship.	

Notation	Name	Description	Example
	Generalization (Inheritance)	Inheritance relationship between classes.	
	Multiplicity	Shows the number of instances that can participate.	
	Public	Visible to all.	
	Private	Visible only within the class.	
	Protected	Visible within the class and its subclasses.	

Identify Classes from the scenario

From the mini scenario:

- Identify 5–7 classes.
- List 3–5 attributes for each class.
- Add at least 2 methods for important classes.
- Add multiplicity.
- Time: 8 mins.

Common Mistakes in Class Diagram

Common errors:

- Only drawing the class name, without attributes/methods.
- Missing multiplicity.
- Confusing relationships with data flows.
- Drawing actors as unnecessary classes.
- Missing main attributes such as purpose, participants, status.
- Relationships that don't reflect business logic.

Sequence Diagram

A sequence diagram shows how actors and objects interact over time.

It focuses on:

- Message order.
- Object interaction.
- Validation logic.
- Service/database calls.
- Return messages.

Mini exercise for Sequence diagram

Draw a sequence diagram for Submit Booking Request.

Must include:

- Actor.
- UI.
- Controller.
- Service.
- Repository/Database.
- Notification.
- Activation bars.
- At least one alt fragment.
- Time: 10 mins

Suggested main flows: Submit Booking Request

- User → Booking UI.
- Booking UI → Booking System
- Booking System → Facility Database.
- Facility Database → Booking Database
- Booking Database → Staff
- Staff → Notification Service

Common Mistakes in Sequence Diagram

Common errors:

- No activation bar.
- Messages not in chronological order.
- Actor calling the database directly.
- No return message.
- No handling of invalid/unavailable cases.
- Too many lifelines making the diagram confusing.
- Inconsistent with the activity diagram.

UI Wireframe & Controls

A UI wireframe is a simple layout of a screen before detailed design.

It shows:

- Screen title.
- Input fields.
- Controls.
- Buttons.
- Tables.
- Messages.
- Navigation.

Controls Selection

Mapping data to controls:

Data	Suitable control
Facility	Dropdown
Date	Date picker
Start time / End time	Time picker
Purpose	Text area
Participants	Number input
Status	Dropdown / Label
Approval decision	Approve / Reject buttons

Validation Rules

Examples:

- Facility is required.
- Date is required.
- Date cannot be in the past.
- Start time must be earlier than end time.
- Participants must be greater than 0.
- Purpose is required.
- Facility must be available.

Feedback Messages

Error message:

- “Please select a facility.”
- “End time must be later than start time.”
- “The selected facility is not available at this time.”

Success message:

- “Booking request submitted successfully.”
- “Your request is pending approval.”

Warning message:

- “This room may not support the expected number of participants.”

Mini Exercise - Design UI Wireframe: Staff Review Form

Must include:

- At least 6 input controls.
- Approve and Reject, Request info buttons.
- Validation messages.
- Success feedback.
- Appropriate control types.
- Time: 10 mins

Common Mistakes in UI Design

Common errors:

- Only listing fields, not designing the layout.
- Incorrect control selection.
- No validation.
- No feedback message.
- No main button.
- Form is too cluttered.
- Missing key information such as purpose, participants, start time, end time.

Course Review Summary

Key takeaway:

To do well in the final exam, students should not memorize diagrams only. They must know how to read a scenario, extract requirements, identify actors and data, model the workflow, model system structure, model object interaction, and design a usable form with proper controls and validation.